

```
$ javac NotificationLab.java && java NotificationLab
```

Java Interface

// □□□□□□□□□□

contract

5/5

dispatch

Page 10 of 10

injection

5/5

```
$ cat learning-path.md
```



01 / NOT COMPLETED

interface NOT COMPLETED implements

02 / NOT COMPLETED

□□□□□□□□

03 / NOT COMPLETED

constructor injection□DIP□ISP

04 / NOT COMPLETED

default□lambda□sealed

□□□□□□□□□□□□□□□□□□□□□□□□

PART 01



□□□□□□□□□□□□

```
1  class NotificationService {
2      void notify(String channel, String message) {
3          if (channel.equals("email")) {
4              new EmailClient().deliver(message);
5          } else if (channel.equals("sms")) {
6              new SmsClient().deliver(message);
7          }
8      }
9  }
```

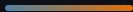
COUPLING REPORT

× □□□□□□□

× □□ SDK □□□□

× □□□□ SDK □□□□

□□ Push → □□□□□□

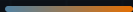


DOMAIN LANGUAGE

Notifier

send(message)





```
1 public interface Notifier {  
2     void send(String message);  
3 }
```

public abstract

□□□□□□□□□□

no instance state

□□□□□□□□□□□□

role name

□□□□□□□□□□

contract ≠ □□□□□□□□□□□□□□□□

implements □□□□□

```
1  final class EmailNotifier implements Notifier {
2      @Override
3      public void send(String message) {
4          System.out.println("EMAIL > " + message);
5      }
6  }
7
8  final class SmsNotifier implements Notifier {
9      @Override
10     public void send(String message) {
11         System.out.println("SMS > " + message);
12     }
13 }
```

□□□□□□□□ public □□□□□□□□□□□□□□□□□ package-private□

PART 02





```
1 Notifier notifier = new EmailNotifier();
2 notifier.send("Welcome");
3
4 notifier = new SmsNotifier();
5 notifier.send("Code: 8042");
```

COMPILE TIME

Notifier 000 send 0

RUN TIME

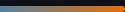
0000000000

00000000 → 00000000



```
1  final class NotificationService {  
2      private final Notifier notifier;  
3  
4      NotificationService(Notifier notifier) {  
5          this.notifier = notifier;  
6      }  
7  
8      void notify(String message) {  
9          notifier.send(message);  
10     }  
11 }
```

□□□□□□□□□□□□□□□□□□□□□□□□



```
1 final class NotificationService {  
2     private final Notifier notifier;  
3  
4     NotificationService(Notifier notifier) {  
5         this.notifier = notifier;  
6     }  
7 }
```

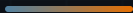
```
final NotificationService
```



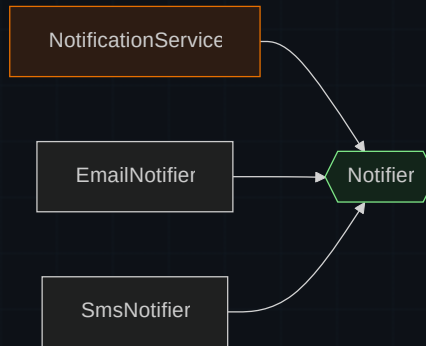
```
1 public static void main(String[] args) {  
2     Notifier notifier =  
3         new EmailNotifier();  
4  
5     var service =  
6         new NotificationService(notifier);  
7 }
```

COMPOSITION ROOT

NotificationService



□□□□□□□□□□□□



□□□□□□□□□□



xxxxxxxxxxxxxxxxxxxxxxxx

xxxx

xxxx

Email

SMS

Push

```
$ deps --graph
```

```
Service → Notifier ← implementations
```

```
xxxx → xx Notifierxxxxxx
```

▶ dispatch()

```
// select channels, then dispatch
```

PART 03



DIP[ISP] [][][][][]

interface ≠ DIP

□□□□

□□□□□□□□□□□□□□□□□□□□

□□□

□□□□□□□□□□□□□□□□□□□□

□□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□

CHECKPOINT_01

00000000 DIP0

A 0000 interface000000 DIP

B 000000000000000000000000

C 00 class 00000000 interface

```
1 interface NotificationPlatform {  
2     void send(String message);  
3     void schedule(String message, Instant at);  
4     void revoke(String id);  
5 }
```

SMS `send` `schedule` `revoke` `UnsupportedOperationException` □□□□□□□□□□

Notifier

Schedulable

Revocable



```
1  final class CompositeNotifier implements Notifier {
2      private final List<Notifier> delegates;
3
4      CompositeNotifier(List<Notifier> delegates) {
5          this.delegates = List.copyOf(delegates);
6      }
7
8      public void send(String message) {
9          for (Notifier notifier : delegates) {
10             notifier.send(message);
11         }
12     }
13 }
```

Notifier



```
1  final class RecordingNotifier
2      implements Notifier {
3
4      final List<String> messages =
5          new ArrayList<>();
6
7      public void send(String message) {
8          messages.add(message);
9      }
10 }
```



```
1  @Test
2  void sendMessage() {
3      var fake = new RecordingNotifier();
4      var service =
5          new NotificationService(fake);
6
7      service.notify("shipped");
8
9      assertEquals(
10         List.of("shipped"),
11         fake.messages);
12 }
```

fast · deterministic · offline

Interface abstract class

	Interface	Abstract class
		
		
		
		
	  default	



PART 04



Java 8 9017 00000

default

NOTES

static

Java8+

```
1 interface Notifier {
2     void send(String message);
3
4     default void sendUrgent(String message) {
5         send("[URGENT] " + message);
6     }
7
8     static Notifier silent() {
9         return message -> { };
10    }
11 }
```

default

API

static

private interface method Java 9+

```
1  interface Notifier {
2      default void sendUrgent(String message) {
3          send("[URGENT] " + normalize(message));
4      }
5
6      default void sendNormal(String message) {
7          send(normalize(message));
8      }
9
10     private String normalize(String message) {
11         return message.strip();
12     }
13
14     void send(String message);
15 }
```

private □□□□□□□□□□□□□□□□

Functional Interface + lambda Java8+

```
1  @FunctionalInterface
2  interface Notifier {
3      void send(String message);
4  }
5
6  Notifier console =
7      message -> System.out.println(message);
```

□□□□□□

default / static / private □□□

@FunctionalInterface

□□□□□□□□□□

Sealed interface Java 17

```
1 sealed interface DeliveryResult
2     permits Delivered, Rejected, Retrying {}
3
4 record Delivered(String id) implements DeliveryResult {}
5 record Rejected(String reason) implements DeliveryResult {}
6 record Retrying(int attempt) implements DeliveryResult {}
```

Sealed interface
Sealed record

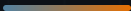
Sealed interface
Sealed record
Sealed class
Sealed enum
Sealed annotation
Sealed module
Sealed package
Sealed plugin API

Java 15 JEP 409 Java 17

Monaco

Java

```
interface Notifier {  
    // TODO: send(String message)  
}  
  
final class PushNotifier implements Notifier {  
    // TODO: "PUSH > " + message  
}  
  
final class NotificationService {  
    // TODO: Notifier  
}
```



01

NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT

02

NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT

03

NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT

04

NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT

05

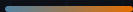
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT
NOT DISRUPT

fake



□□	□□	□□
default / static methods	Java 8	□□□□□□□□
Functional interface / lambda	Java 8	□□□□□□□□
private interface methods	Java 9	□□□□□□
Sealed interfaces	Java 17	□□□□□□□□

□□□□JLS §9 · JLS §9.8 · JEP 409



□□□□□□□□□□



□□□□□□□□□□



□□□□□□□□□□



NOT DESIGNING
NOT DESIGNING

fake NOT DESIGNING □□□□□□

Interface □□□□□□□□□□□□□□

Lab complete.

~~~~~ FailingNotifier ~~~~~

```
$ java NotificationLab --reflect
```

```
contract → dispatch → injection → test
```